Yesterday 10:30 PM

Assignment - Python_Dja...
Document

Prompt-1 Requirement Analysis

I am working on a Django backend hiring assignment.

I want to understand the problem thoroughly before writing any code.

Read the assignment and help me identify:

1. Functional requirements
2. Business rules

Show more

Based on the assignment, the core problem is to build an **AI-assisted box recommendation system** for an e-commerce warehouse. The objective is to recommend the most suitable shipping box for a customer's order based on product dimensions, weight, available box specifications, and cost. Assignment - Python_Django Deve...

---

1. Functional Requirements

Product Management

The system should maintain products with:

- Length
- Width
- Height
- Weight

These values will later be used while calculating packaging suitability.

Assignment - Python_Django Deve...

Box Management

The system should maintain shipping boxes with:

- Internal Length
- Internal Width
- Internal Height
- Maximum supported weight
- Cost

Every box represents a possible packaging option. Assignment - Python_Django Deve...

Order Handling

An order contains one or more products.

The recommendation engine should receive an order and determine the most suitable shipping box.

Recommendation Engine

For every order:

- Determine which boxes can physically fit the products.
- Ensure the total product weight does not exceed the box capacity.
- From all valid boxes, recommend the best one.

The assignment only says "most suitable", so the selection strategy must be defined by the engineer. Assignment - Python_Django Deve...

Testing

The submission explicitly requires test cases.

That means the recommendation logic should be easily testable with unit tests.

Assignment - Python_Django Deve...

2. Business Rules

The assignment explicitly gives some business constraints.

Rule 1

A product must physically fit inside the box.

Meaning:

```
product_length <= box_internal_length
product_width  <= box_internal_width
product_height <= box_internal_height
```

Rule 2

The total weight must not exceed the box capacity.

```
sum(product_weights) <= box.max_weight
```

Rule 3

Every recommendation must choose an existing box.

No custom/generated boxes.

Rule 4

The chosen box should be the "most suitable."

This is intentionally vague.

Possible interpretations:

- Cheapest valid box
- Smallest valid box
- Least unused volume

- Best cost efficiency

The assignment leaves this open.

Rule 5

Warehouse employees need exactly one recommendation.

The wording suggests the engine returns one best box rather than all possible boxes.

---

3. Hidden Assumptions

These are things the assignment does not state but you'll need to decide.

Assumption A

Can products be rotated?

Example:

Product

```
20 × 30 × 10
```

Can it become

```
30 × 20 × 10
```

Most packing systems allow rotation.

The assignment never specifies this.

Assumption B

How are multiple products packed?

Example:

```
Phone
Laptop
Bottle
```

Do we:

- stack them?
- place them side-by-side?
- calculate combined dimensions?
- assume perfect packing?

Packing multiple objects is actually a difficult optimization problem.

The assignment completely ignores this.

Assumption C

Can products overlap?

Obviously no.

But how is packing validated?

No instructions are given.

Assumption D

Do box dimensions represent:

- internal dimensions
- external dimensions

Fortunately, the assignment says **internal dimensions**, so packing calculations should use internal measurements. Assignment - Python_Django Deve...

Assumption E

Can products be fragile?

No mention.

Ignore.

Assumption F

Does empty space matter?

Example

Product volume

```
900 cm³
```

Box volume

```
20,000 cm³
```

Should such a huge box be selected?

Probably not.

Assumption G

If two boxes satisfy everything, how do we break the tie?

Need a deterministic rule.

For example:

- lowest cost
- then smallest volume
- then lowest ID

---

4. Edge Cases

A backend engineer should think about unusual inputs before coding.

Empty Order

No products.

Should return:

- validation error
- no recommendation

Product Too Heavy

Product weight exceeds every box capacity.

Expected:

No suitable box.

Product Too Large

Dimensions exceed every available box.

Expected:

No suitable box.

Multiple Valid Boxes

Example

```
Small Box
Medium Box
Large Box
```

All fit.

Which one wins?

Exact Fit

Product dimensions exactly equal box dimensions.

Should this count as valid?

Usually yes.

Exact Weight Limit

```
Total Weight = Max Weight
```

Should still be valid.

Duplicate Products

Order contains

```
Laptop × 5
```

Need to decide whether dimensions and weight calculations treat quantity correctly.

Missing Dimensions

```
length = NULL
```

Should be rejected during validation.

Zero Dimensions

```
Length = 0
```

Invalid.

Negative Values

```
Weight = -5
```

Invalid.

Floating Point Values

```
12.4 cm
```

Need consistent precision rules.

No Boxes in Database

Recommendation should fail gracefully.

No Products in Database

Order creation should fail.

---

5. Possible Ambiguities

These are the biggest areas where the assignment leaves room for engineering decisions.

Ambiguity 1

What exactly means "most suitable"?

No definition.

Ambiguity 2

How should multiple products be packed?

Real packing is NP-hard.

Assignment doesn't specify.

Ambiguity 3

Can products rotate?

No answer.

Ambiguity 4

Should recommendation consider:

- box cost
- box volume
- shipping efficiency

Only cost is mentioned as a box attribute; no optimization rule is defined.

Assignment - Python_Django Deve...

Ambiguity 5

Should one order use multiple boxes?

The wording implies a single recommended box, but it doesn't explicitly state whether splitting an order across boxes is allowed.

Ambiguity 6

How should ties be resolved?

No tie-breaking strategy is specified.

Questions I would ask the interviewer

Before implementation, I'd clarify these points:

1. Can products be rotated to fit inside a box?
2. Should an order always be packed into a single box?
3. How should multiple products be arranged within a box?
4. What defines the "most suitable" box (lowest cost, smallest volume, least unused space, etc.)?

5. If multiple boxes are equally suitable, what tie-breaking rule should be used?
6. Should the system return a recommendation failure when no box fits, or suggest an alternative workflow?

These questions demonstrate backend engineering thinking by identifying requirements gaps before designing the solution.

Sources

Today 3:18 PM

Prompt5- Recommendation Algorithm

I need to design an algorithm that recommends the best shipping box for an order.

The recommendation should consider:

- dimensions
- weight
- box capacity
- cost

Show more

The assignment asks you to recommend the **most suitable shipping box** for an order based on **product dimensions, weight, box capacity, and cost**. It does **not** prescribe a specific algorithm, so your design should be simple, deterministic, explainable, and easy to test—qualities that are important in a backend hiring assignment.

Assignment - Python_Django Deve...

Step 1: Define the Inputs

The algorithm receives:

Order

- One or more products
- Each product has:
  - Length

- Width
- Height
- Weight

Available Boxes

Each box has:

- Internal Length
- Internal Width
- Internal Height
- Maximum Weight
- Cost

The output should be:

- One recommended box
- OR
- "No suitable box"

Step 2: Validate the Order

Before recommending anything, validate the data.

Questions to check:

- Does the order contain at least one product?
- Are dimensions positive?
- Is weight positive?
- Are values missing?

If validation fails:

> Return an appropriate validation error.

This keeps recommendation logic separate from input validation.

Step 3: Calculate Order Requirements

Next determine what the order requires.

There are two independent constraints.

A. Weight Requirement

Compute

```
Total Order Weight
=
sum(all product weights)
```

Example

```
Phone = 0.3 kg
Laptop = 2.2 kg
Mouse = 0.1 kg

Total = 2.6 kg
```

B. Dimension Requirement

This is where the assignment becomes ambiguous.

For multiple products there are several possible approaches.

Approach 1 — Bounding Box (Recommended)

Imagine wrapping all products inside one invisible rectangular box.

Example

```
Product A

20 × 10 × 5

Product B

15 × 8 × 4
```

Estimate the required dimensions by combining them according to a simple packing assumption.

Pros

- Easy to implement
- Deterministic
- Easy to explain
- Easy to test

Cons

- Doesn't produce the optimal packing.

For a hiring assignment, this trade-off is usually acceptable because it avoids unnecessary complexity.

Approach 2 — Sum One Dimension

Assume products are placed in a row.

Example

```
Length

20 + 15 = 35

Width

max(10,8)=10

Height

max(5,4)=5
```

Very easy.

But often wastes space.

Approach 3 — Total Volume Only

Calculate

```
Total Product Volume
```

Then compare against

```
Box Volume
```

Problem

A product can have smaller volume yet still not fit because of its shape.

Example

```
Product

100 × 2 × 2
```

may not fit inside

```
20 × 20 × 20
```

even though the volumes are compatible.

Volume alone is therefore insufficient.

Approach 4 — 3D Bin Packing

This is how Amazon-like systems work.

The algorithm considers

- rotations
- orientations
- free spaces
- placement optimization

Pros

Highly accurate.

Cons

- Complex
- Time consuming
- Hard to debug
- Overkill for this assignment

Recommendation

Use the **Bounding Box / Simple Packing Assumption** because it is:

- understandable
- deterministic
- easy to justify in documentation
- appropriate for the assignment's scope

Step 4: Filter Invalid Boxes

Now inspect every available box.

A box is rejected if:

Weight exceeds capacity

```
Order Weight

>

Box Capacity
```

Reject.

Dimensions don't fit

If required dimensions exceed the box's internal dimensions.

Reject.

Remaining boxes become candidates.

Step 5: Rank Candidate Boxes

Usually multiple boxes will qualify.

Example

Box	Fits	Cost
Small	✅	₹50
Medium	✅	₹60
Large	✅	₹90

Now decide which is "best."

Strategy 1 — Cheapest Box

Choose the lowest-cost valid box.

Advantages

- Saves shipping cost
- Very easy
- Easy to explain
- Deterministic

Strategy 2 — Smallest Volume

Choose the smallest box that fits.

Advantages

- Less empty space
- Better packaging efficiency

Disadvantage

May cost more.

Strategy 3 — Least Wasted Space

Calculate

```
Unused Volume

=

Box Volume

-

Required Volume
```

Choose the minimum.

Advantages

Efficient.

Disadvantages

Still ignores product shape.

Strategy 4 — Weighted Score

Example

```
Score

=

0.7 × Cost

+

0.3 × Empty Space
```

Lower score wins.

Useful in real systems.

Not necessary here.

Recommended Ranking Strategy

For this assignment, use the following priority:

1. Must fit dimensions.
2. Must satisfy weight limit.
3. Lowest cost.
4. If costs are equal, choose the smaller box (least internal volume).
5. If still tied, choose a consistent fallback (such as the lowest box ID or name).

This yields predictable, testable behavior.

Step 6: Return Recommendation

Possible outcomes:

Case 1

Exactly one valid box

Return it.

Case 2

Several valid boxes

Return the highest-ranked one.

Case 3

No valid box

Return a clear "No suitable box found" result rather than forcing an invalid recommendation.

Time Complexity

Let:

- **n** = number of products
- **m** = number of boxes

Then:

- Aggregate order requirements: **O(n)**
- Check each box: **O(m)**

Overall complexity:

O(n + m)

This is efficient and scales well for typical warehouse operations.

Design Principles

A clean backend design benefits from separating concerns:

1. **Validation** – ensure product and box data are valid.
2. **Requirement Calculation** – compute the order's weight and required dimensions.
3. **Candidate Filtering** – eliminate boxes that cannot satisfy the constraints.
4. **Ranking** – apply deterministic business rules to choose the best candidate.
5. **Recommendation** – return the selected box or indicate that no suitable box exists.

Why this approach fits the assignment

Compared with more sophisticated packing algorithms, this approach is the best balance for a hiring exercise because it:

- Correctly enforces the stated constraints (dimensions, weight, capacity, and cost).
  Assignment - Python_Django Deve...
- Is straightforward to explain during a code review.
- Produces deterministic, repeatable results for unit tests.
- Avoids unnecessary complexity such as full 3D bin packing while leaving room to document assumptions and future improvements.

Sources

Today 4:53 PM

Prompt-15 final engineering review

Pretend you are a senior backend engineer reviewing this assignment before it is submitted.

Evaluate the project on:

- Architecture
- Code Quality
- Readability
- Testing

Show more

Based on what you've shared in this conversation, I **can't honestly review the implementation itself** because I haven't seen the project's source code, tests, API, or documentation. What I *can* do is evaluate your **overall engineering approach and design decisions** against what this assignment asks for. Assignment - Python_Django Deve...

If I were reviewing you as a senior backend engineer before submission, here's how I would score your project **assuming the implementation follows the design we've discussed**.

Category	Score	Review
Architecture	**9/10**	The proposed separation of models, recommendation logic, validation, and API responsibilities demonstrates good modular design. Business logic is planned outside views, which is a strong backend practice.
Code Quality	**8.5/10**	The planned solution emphasizes clear business rules, deterministic behavior, and maintainability. The final score would depend on consistent naming, small functions, and avoiding duplicated logic.
Readability	**9/10**	The engineering-first approach, documented assumptions, and straightforward recommendation algorithm should make the code easy to understand for reviewers.
Testing	**8.5/10**	The assignment explicitly requires test cases. If you cover happy paths, invalid input, boundary conditions, and "no suitable box" scenarios, this would be a strong submission. Assignment - Python_Django Deve...
API Design	**8.5/10**	A REST API with proper request validation, meaningful HTTP status codes, and consistent response formats would satisfy expectations.

Category	Score	Review
		Including informative error messages would strengthen it further.
Documentation	**9.5/10**	If you provide a clear README, AI usage report, assumptions, setup instructions, API examples, testing instructions, and engineering decisions, your documentation would likely stand out. The assignment specifically asks for README, AI usage documentation, test output, and lessons learned. Assignment - Python_Django Deve...

Overall Assessment

Overall Score: 8.9/10

From an interviewer's perspective, this would be a **strong junior/backend developer submission**, particularly because you've focused on understanding the problem before implementing it.

Three Strongest Aspects

1. Engineering-First Thinking

Instead of immediately writing code, you analyzed:

- business rules
- assumptions
- edge cases
- ambiguities

That reflects how backend engineers approach real projects.

---

2. Deterministic Recommendation Logic

Your proposed algorithm is:

- simple
- explainable
- testable
- deterministic

Those qualities are often preferred over unnecessarily complex solutions in hiring assignments.

---

3. Clear Separation of Concerns

Your planned design separates:

- validation
- recommendation logic
- data models
- API layer
- testing

This improves maintainability and makes future changes easier.

Three Improvements That Would Strengthen the Submission

1. Explicitly Document Assumptions

The assignment leaves several behaviors unspecified (such as product rotation, packing strategy, and tie-breaking). A dedicated **Assumptions** section in the README explaining these design decisions would help reviewers understand your reasoning and avoid guessing.

---

2. Include Comprehensive Edge-Case Tests

Beyond standard success cases, include tests for:

- empty orders
- overweight orders
- oversized products
- exact dimension matches
- exact weight limits
- multiple valid boxes
- no valid boxes
- invalid dimensions or weights

This demonstrates attention to robustness.

---

3. Explain Algorithm Trade-offs

Briefly describe why you chose a simple deterministic approach instead of full 3D bin packing:

- lower complexity
- easier maintenance
- predictable behavior
- appropriate for assignment scope

Showing that you considered alternatives demonstrates engineering judgment.

Hiring Decision

If the implementation matches the architecture and reasoning we've discussed, my evaluation would be:

Technical Competency: ✅ Strong for a junior/backend role

Problem Solving: ✅ Above average

Code Organization: ✅ Good

Communication & Documentation: ✅ Strong

Recommendation: Hire for the next interview stage.

The main reason would not be the complexity of the algorithm itself, but the fact that you approached the problem methodically, made explicit design decisions, and aimed for a solution that is maintainable, testable, and well documented—all qualities that matter in backend engineering.